

```

"""Lightweight connection handling for MCP servers."""

from abc import ABC, abstractmethod
from contextlib import AsyncExitStack
from typing import Any

from mcp import ClientSession, StdioServerParameters
from mcp.client.sse import sse_client
from mcp.client.stdio import stdio_client
from mcp.client.streamable_http import streamablehttp_client

class MCPConnection(ABC):
    """Base class for MCP server connections."""

    def __init__(self):
        self.session = None
        self._stack = None

    @abstractmethod
    def _create_context(self):
        """Create the connection context based on connection type."""

    async def __aenter__(self):
        """Initialize MCP server connection."""
        self._stack = AsyncExitStack()
        await self._stack.__aenter__()

        try:
            ctx = self._create_context()
            result = await self._stack.enter_async_context(ctx)

            if len(result) == 2:
                read, write = result
            elif len(result) == 3:
                read, write, _ = result
            else:
                raise ValueError(f"Unexpected context result: {result}")

            session_ctx = ClientSession(read, write)
            self.session = await self._stack.enter_async_context(session_ctx)
            await self.session.initialize()
            return self
        except BaseException:
            await self._stack.__aexit__(None, None, None)
            raise

    async def __aexit__(self, exc_type, exc_val, exc_tb):
        """Clean up MCP server connection resources."""
        if self._stack:
            await self._stack.__aexit__(exc_type, exc_val, exc_tb)
        self.session = None
        self._stack = None

    async def list_tools(self) -> list[dict[str, Any]]:

```

```

        """Retrieve available tools from the MCP server."""
        response = await self.session.list_tools()
        return [
            {
                "name": tool.name,
                "description": tool.description,
                "input_schema": tool.inputSchema,
            }
            for tool in response.tools
        ]

    async def call_tool(self, tool_name: str, arguments: dict[str, Any]) -> Any:
        """Call a tool on the MCP server with provided arguments."""
        result = await self.session.call_tool(tool_name, arguments=arguments)
        return result.content

class MCPConnectionStdio(MCPConnection):
    """MCP connection using standard input/output."""

    def __init__(self, command: str, args: list[str] = None, env: dict[str, str] = None):
        super().__init__()
        self.command = command
        self.args = args or []
        self.env = env

    def _create_context(self):
        return stdio_client(
            StdioServerParameters(command=self.command, args=self.args, env=self.env)
        )

class MCPConnectionSSE(MCPConnection):
    """MCP connection using Server-Sent Events."""

    def __init__(self, url: str, headers: dict[str, str] = None):
        super().__init__()
        self.url = url
        self.headers = headers or {}

    def _create_context(self):
        return sse_client(url=self.url, headers=self.headers)

class MCPConnectionHTTP(MCPConnection):
    """MCP connection using Streamable HTTP."""

    def __init__(self, url: str, headers: dict[str, str] = None):
        super().__init__()
        self.url = url
        self.headers = headers or {}

    def _create_context(self):
        return streamablehttp_client(url=self.url, headers=self.headers)

```

```

def create_connection(
    transport: str,
    command: str = None,
    args: list[str] = None,
    env: dict[str, str] = None,
    url: str = None,
    headers: dict[str, str] = None,
) -> MCPConnection:
    """Factory function to create the appropriate MCP connection.

    Args:
        transport: Connection type ("stdio", "sse", or "http")
        command: Command to run (stdio only)
        args: Command arguments (stdio only)
        env: Environment variables (stdio only)
        url: Server URL (sse and http only)
        headers: HTTP headers (sse and http only)

    Returns:
        MCPConnection instance
    """
    transport = transport.lower()

    if transport == "stdio":
        if not command:
            raise ValueError("Command is required for stdio transport")
        return MCPConnectionStdio(command=command, args=args, env=env)

    elif transport == "sse":
        if not url:
            raise ValueError("URL is required for sse transport")
        return MCPConnectionSSE(url=url, headers=headers)

    elif transport in ["http", "streamable_http", "streamable-http"]:
        if not url:
            raise ValueError("URL is required for http transport")
        return MCPConnectionHTTP(url=url, headers=headers)

    else:
        raise ValueError(f"Unsupported transport type: {transport}. Use 'stdio', 'sse', or 'http'")

```